Loops and conditionals include constructs such as `if` and `for` loops. Using them in Razor templates is almost identical to C#, but you need to prefix their usage with the `@` symbol. In case you're not getting the hang of Razor yet, when in doubt, throw in another `@`!

One of the big advantages of Razor in the context of ASP.NET Core is that it uses languages you're already familiar with: C# and HTML. There's no need to learn a whole new set of primitives for some other templating language: it's the same `if`, `foreach`, and `while` constructs you already know. And when you don't need them, you're writing raw HTML, so you can see exactly what the user is getting in their browser.

In listing 17.6, I've applied a few of these techniques in a template to display a to-do item. The `PageModel` has a `bool IsComplete` property, as well as a `List<string>` property called `Tasks`, which contains any outstanding tasks.

Listing 17.6 Razor template for rendering a `ToDoItemViewModel`

```
@page
@model ToDoItemModel                                    ❶
<div>
    @if (Model.IsComplete)
    {                                                   ❷
        <strong>Well done, you're all done!</strong>    ❷
    }                                                   ❷
    else
    {
        <strong>The following tasks remain:</strong>
        <ul>
            @foreach (var task in Model.Tasks)          ❸
            {
                <li>@task</li>                          ❹
            }
        </ul>
    }
</div>
```

❶ The @model directive indicates the type of PageModel in Model.